

cache strategies

Практика: сравнение типов кеширования

Цель работы — сравнить три стратегии кеширования на одной системе и одном тесте

- Lazy Loading / Cache-Aside / Write-Around
- Write-Through
- Write-Back

Стенд

- load-generator: Python-скрипт bench.py
- application: тот же скрипт, внутри есть единый слой работы с данными
- cache: Redis
- БД: PostgreSQL

В каждом прогоне используется одинаковый набор данных: 1000 записей в таблице items

Перед каждым тестом БД заполняется заново, Redis очищается

Запуск

```
docker compose up -d postgres redis

docker compose build runner

docker compose run --rm runner bash scripts/run_all.sh
```

Результаты сохраняются в results/results.csv

```
Container cache-strategies-comparison-runner-run-f90cece68f48 Creating
Container cache-strategies-comparison-runner-run-f90cece68f48 Created
{'strategy': 'cache-aside', 'workload': 'read-heavy', 'read_percent': 80, 'write_percent': 20, 'requests': 12000, 'elapsed_sec': 15.0, 'throughput_rps': 800.04, 'avg_ms': 0.74, 'p95_ms': 1.71, 'db_reads': 2588, 'db_writes': 2428, 'db_calls': 5016, 'cache_hits': 6984, 'cache_misses': 2588, 'hit_rate': 0.7296, 'pending_writes': 0, 'max_pending_writes': 0, 'errors': 0}
{'strategy': 'cache-aside', 'workload': 'balanced', 'read_percent': 50, 'write_percent': 50, 'requests': 12000, 'elapsed_sec': 15.0, 'throughput_rps': 800.04, 'avg_ms': 1.16, 'p95_ms': 1.83, 'db_reads': 3255, 'db_writes': 6036, 'db_calls': 9291, 'cache_hits': 2709, 'cache_misses': 3255, 'hit_rate': 0.4542, 'pending_writes': 0, 'max_pending_writes': 0, 'errors': 0}
{'strategy': 'cache-aside', 'workload': 'write-heavy', 'read_percent': 20, 'write_percent': 80, 'requests': 12000, 'elapsed_sec': 18.47, 'throughput_rps': 649.57, 'avg_ms': 1.52, 'p95_ms': 2.24, 'db_reads': 1976, 'db_writes': 9536, 'db_calls': 11512, 'cache_hits': 488, 'cache_misses': 1976, 'hit_rate': 0.1981, 'pending_writes': 0, 'max_pending_writes': 0, 'errors': 0}
{'strategy': 'write-through', 'workload': 'read-heavy', 'read_percent': 80, 'write_percent': 20, 'requests': 12000, 'elapsed_sec': 15.0, 'throughput_rps': 800.05, 'avg_ms': 0.68, 'p95_ms': 1.83, 'db_reads': 783, 'db_writes': 2428, 'db_calls': 3211, 'cache_hits': 8789, 'cache_misses': 783, 'hit_rate': 0.9182, 'pending_writes': 0, 'max_pending_writes': 0, 'errors': 0}
{'strategy': 'write-through', 'workload': 'balanced', 'read_percent': 50, 'write_percent': 50, 'requests': 12000, 'elapsed_sec': 15.0, 'throughput_rps': 800.02, 'avg_ms': 1.02, 'p95_ms': 1.91, 'db_reads': 479, 'db_writes': 6036, 'db_calls': 6515, 'cache_hits': 5485, 'cache_misses': 479, 'hit_rate': 0.9197, 'pending_writes': 0, 'max_pending_writes': 0, 'errors': 0}
{'strategy': 'write-through', 'workload': 'write-heavy', 'read_percent': 20, 'write_percent': 80, 'requests': 12000, 'elapsed_sec': 16.25, 'throughput_rps': 738.35, 'avg_ms': 1.34, 'p95_ms': 1.94, 'db_reads': 216, 'db_writes': 9536, 'db_calls': 9752, 'cache_hits': 2248, 'cache_misses': 216, 'hit_rate': 0.9123, 'pending_writes': 0, 'max_pending_writes': 0, 'errors': 0}
{'strategy': 'write-back', 'workload': 'read-heavy', 'read_percent': 80, 'write_percent': 20, 'requests': 12000, 'elapsed_sec': 15.0, 'throughput_rps': 800.03, 'avg_ms': 0.4, 'p95_ms': 0.98, 'db_reads': 783, 'db_writes': 100, 'db_calls': 883, 'cache_hits': 8789, 'cache_misses': 783, 'hit_rate': 0.9182, 'pending_writes': 881, 'max_pending_writes': 881, 'errors': 0}
```

Описание тестов

Все стратегии прогонялись на одних параметрах

- requests: 12000
- duration: 15 секунд
- items: 1000
- producer/load-generator: 1 процесс
- cache: Redis single instance
- DB: PostgreSQL single instance

Профили нагрузки

- read-heavy: 80% read / 20% write
- balanced: 50% read / 50% write
- write-heavy: 20% read / 80% write

Метрики

- throughput_rps — фактическая пропускная способность
- avg_ms — средняя задержка
- db_calls — обращения в БД во время активного теста
- hit_rate — доля чтений, которые попали в кеш
- pending_writes — накопленные записи Write-Back перед финальным сбросом

После замера Write-Back сбрасывает накопленные записи в БД, чтобы следующий тест стартовал с чистого состояния

Таблица результатов

strategy	workload	read/write	requests	elapsed sec	throughput rps	avg ms	p95 ms	db calls	cache hits	cache misses	hit rate	pending writes	max pending writes	errors
Cache-Aside	read-heavy	80/20	12000	15.0	800.03	0.73	1.67	5016	6984	2588	72.96%	0	0	0
Cache-Aside	balanced	50/50	12000	15.0	800.03	1.14	1.75	9291	2709	3255	45.42%	0	0	0
Cache-Aside	write-heavy	20/80	12000	17.2	697.56	1.42	1.84	11512	488	1976	19.81%	0	0	0
Write-Through	read-heavy	80/20	12000	15.0	800.04	0.65	1.76	3211	8789	783	91.82%	0	0	0
Write-Through	balanced	50/50	12000	15.0	800.01	1.0	1.82	6515	5485	479	91.97%	0	0	0
Write-Through	write-heavy	20/80	12000	16.24	738.82	1.34	1.88	9752	2248	216	91.23%	0	0	0
Write-Back	read-heavy	80/20	12000	15.0	800.03	0.4	0.96	883	8789	783	91.82%	872	872	0
Write-Back	balanced	50/50	12000	15.0	800.03	0.47	0.83	579	5485	479	91.97%	987	987	0
Write-Back	write-heavy	20/80	12000	15.0	800.02	0.52	0.69	316	2248	216	91.23%	997	997	0

Выводы

Для чтения лучше всего показали себя Write-Through и Write-Back

У них hit rate около 92%, потому что запись сразу обновляет кеш, и следующие чтения чаще берут данные из Redis

Cache-Aside хуже держит hit rate на смешанной и write-heavy нагрузке, потому что запись идет в БД и инвалидирует кеш

Для записи быстрее всего оказался Write-Back

Он не пишет каждое изменение сразу в PostgreSQL, поэтому средняя задержка на write-heavy составила 0.52 ms против 1.34 ms у Write-Through и 1.42 ms у Cache-Aside

Для смешанной нагрузки лучшим вариантом по задержке и числу обращений в БД стал Write-Back

На balanced нагрузке у него 579 обращений в БД, у Write-Through 6515, у Cache-Aside 9291

Главный минус Write-Back — накопление записей

На write-heavy перед финальным сбросом накопилось 997 pending writes, значит при падении приложения без надежной очереди часть свежих изменений можно потерять

Если важна простота и запись должна сразу быть в БД, лучше выбрать Write-Through

Если главное — скорость записи и можно отдельно защищать от потери pending writes, лучше подходит Write-Back